

SIMATS ENGINEERING

ASSESSMENT TOOL 2

NAME:K.Deepika

REG NO: 192371042

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4303

C04-AT2

Question 1: XML Parsing Algorithm

Aim: To parse an XML document containing structured user or product data, traverse its hierarchical nodes, extract required values, and display the data in readable form.

Pseudocode Algorithm:

ALGORITHM XML_PARSING

START

1. Specify the path of the XML file.
2. Check whether the XML file exists.

IF file does not exist THEN

Display "XML file not found."

STOP

END IF

3. Load the XML document into an XML parser.
4. Parse the XML document.
5. Check whether the XML is well-formed.

IF parsing fails THEN

Display "Invalid XML document."

STOP

END IF

6. Get the root node of the XML document.

7. Display the root element name.
8. Traverse each child node of the root element.
9. FOR each child node DO
 - IF the node contains nested child elements THEN
 - Traverse the nested nodes.
 - END IF
 - Identify the required element names.
 - Extract their text values or attributes. Store the extracted values.
- END FOR
10. Display the extracted information in a readable format.
11. Close or release the parser resources.

END

Example Output:

Product ID: P101

Product Name: Laptop

Category: Electronics

Price: ₹55000

Question 2: MVC Workflow

Aim: To represent how a user request moves through the Controller, Model, database, and View before the final response is returned.

Pseudocode Algorithm:

ALGORITHM

MVC_WORKFLOW

START

1. User enters a URL or submits a form.
2. Browser sends the HTTP request to the web server.

3. Server identifies the appropriate Controller.
4. Controller receives and analyzes the request.
5. Controller validates the request parameters.

IF input is invalid THEN

Controller sends an error response to the View.

Display error message.

STOP

END IF

6. Controller calls the required Model or Service.
7. Model processes the business logic.
8. Model establishes a connection with the database.
9. Model executes the required database operation.
10. Database returns the requested data.
11. Model processes the returned data.
12. Model sends the result back to the Controller.
13. Controller selects the appropriate View.
14. Controller passes the model data to the View.
15. View generates the dynamic HTML response.
16. Server sends the response to the browser.
17. Browser displays the result to the user.

END

MVC Flow:

User → Controller → Model → Database → Model → Controller → View → User

Question 3: JSP Request Flow

Aim: To describe the complete request–response cycle in a JSP-based web application, including client requests, Servlet/JSP processing, backend interaction, and dynamic response generation.

Pseudocode Algorithm:

ALGORITHM JSP_REQUEST_RESPONSE

START

1. User enters a URL or submits a form from the browser.
2. Browser creates an HTTP request.
3. HTTP request is sent to the web server/application server.
4. Server identifies the requested JSP resource.
5. Check whether the JSP has already been translated and compiled.

IF JSP is not compiled or has been modified THEN

Translate JSP into a Servlet.

Compile the generated Servlet.

END IF

6. Container executes the JSP Servlet.
7. JSP/Servlet reads request parameters.
8. Validate the received input.

IF input is invalid THEN

Generate an error message.

Send the response to the client.

STOP

END IF

9. JSP/Servlet calls the required backend component.
10. Backend component performs business logic.
11. IF database data is required
THEN Establish database
connection. Execute SQL query.
Retrieve the result.
Close/release database resources.
END IF

12. Pass the processed data to the JSP View.
13. JSP generates dynamic HTML content.
14. Web server creates the HTTP response.
15. Send the response back to the browser.

16. Browser renders and displays the dynamic page.

END

JSP Request Flow:

**Client → Web Server → JSP/Servlet → Backend/Database → JSP View → HTTP
Response → Client**

Question 4: Database Retrieval Algorithm

Aim: To retrieve records from a database by establishing a connection, executing an SQL query, processing the result set, and displaying the retrieved information.

Pseudocode Algorithm:

ALGORITHM DATABASE_RETRIEVAL

START

1. Read the required search criteria from the user.
2. Load the required database driver or database library.
3. Specify the database URL, username, and password.
4. Establish a connection with the database.

5. Check whether the connection is successful.

IF connection fails THEN

Display "Database connection failed."

STOP

END IF

6. Create a prepared statement.
7. Write the required SQL SELECT query.
8. Bind user-provided values to query parameters.
9. Execute the SELECT query.
10. Store the returned records in a Result Set.
11. Check whether the Result Set contains records.

```

IF no records are found THEN
    Display "No records
    found."
ELSE
    Display the table headings.
    WHILE another record exists DO
        Read     each     required
        column. Store or process
        the values. Display the
        record.
    END
WHILE END
IF

```

12. Close the Result Set.
13. Close the prepared statement.
14. Close the database connection.
15. Display the final retrieved data.

END

Database Retrieval Flow:

**Connect → Prepare SQL → Execute Query → Receive Result Set → Process Records
→ Display → Close Resources**

Question 5: PHP Validation Algorithm

Aim: To validate user input submitted through a web form using PHP concepts and allow submission only when all required validation checks are satisfied.

Pseudocode Algorithm:

ALGORITHM PHP_FORM_VALIDATION

START

1. Receive the submitted form data.
2. Initialize an empty error list.
3. Read the name, email, password, and confirm-password fields.

4. Remove unnecessary spaces from input values.

5. Check whether the name field is empty.

IF name is empty THEN

Add "Name is required" to the error list.

END IF

6. Check whether the email field is empty.

IF email is empty THEN

Add "Email is required" to the error list.

ELSE

Check whether the email has a valid format.

IF email format is invalid THEN

Add "Enter a valid email address" to the error list.

END

IF END

IF

7. Check whether the password field is empty.

IF password is empty THEN

Add "Password is required" to the error list.

ELSE

Check whether password length meets the required minimum.

IF password is too short THEN

Add "Password must meet the minimum length" to the error list.

END

IF END

IF

8. Check whether confirm-password is equal to password.

IF passwords do not match THEN

Add "Passwords do not match" to the error list.

END IF

9. Check the error list.

IF errors exist

THEN

Display each appropriate error message.

Keep the form open for correction.

ELSE

Sanitize/prepare the validated data.

Process or store the data.

Display "Form submitted successfully."

END IF

END

Overall Conclusion

The five algorithms demonstrate the major processing stages of a data-driven web application. XML parsing enables structured data to be loaded, traversed, and extracted. MVC separates the Controller, Model, and View to improve organization and maintainability. JSP request processing shows how a client request is converted into dynamic server-generated content. Database retrieval provides a systematic method for connecting to a database, executing SQL, processing results, and releasing resources. PHP validation ensures that user input is checked before it is processed or stored. Together, these algorithms provide a clear foundation for designing reliable XML-, JSP-, MVC-, database-, and PHP-based web solutions.

Code of Conduct:

I _____(K.Deepika/ 192371042) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K.Deepika

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient